Université Paris Cité

Compléments en Programmation Orientée Objet
Projet de session 2 : Calculatrice à ensembles

Projet à réaliser seul.

1 Avant-propos

Le projet consiste en l'implémentation d'un logiciel de calcul d'ensembles d'entiers, à l'aide de constantes entières et d'opérations (entiers vers entiers, entiers vers ensemble et ensemble vers ensemble.)ns
Nous insistons ici sur la conception objet (abstractions objet pertinentes et patrons de conception). Il ne suffira pas de rendre un programme qui fonctionne pour avoir une bonne note ! Par ailleurs, attendez-vous à des questions concernant vos choix techniques et soyez prêts à les défendre à l'oral.

2 Instructions de rendu

- Vous travaillerez sur un dépôt git privé hébergé sur le serveur `https://moule.informatique.univ-paris-diderot.fr` et veillerez à faire des commits réguliers.
- Vous donnerez l'accès à votre dépôt (notamment à son historique, dans le doute choisissez *maintainer*) aux enseignants du module.
- Vous déposerez sur moodle un lien vers votre git (dans un fichier texte). [1].
- Le point d'entrée de votre documentation sera le fichier `README.md` situé à la racine de votre dépôt dans la branche par défaut.
- Inscrivez-vous pour votre oral/soutenance dès que le lien est disponible.

3 Le modèle de calcul : calculatrice en notation polonaise inverse (RPN)

On part ici sur une calculatrice en notation polonaise inverse (RPN) : cela a l'avantage d'éviter d'implémenter un analyseur syntaxique.
Le principe est le suivant : la mémoire de la calculatrice est constituée d'une pile LIFO d'éléments du domaine de calcul (typiquement des nombres ; dans notre cas, des ensembles de nombres ; pour l'exemple, considérons que les opérandes sont des nombres). Une boucle de saisie permet à l'utilisateur d'entrer des opérations successivement :

- des opérations 0-aires, c'est-à-dire des opérandes (des nombres) : l'opérande est ajoutée à la pile
- des opérations unaires, c'est-à-dire des fonctions (des nombres vers les nombres) à un argument : 1 opérande est retirée de la pile, la fonction lui est appliquée, et le résultat, qui est lui-même une opérande, est empilé.
- des opérations binaires : 2 opérandes sont retirées, l'opération leur est appliquée, le résultat est empilé
- et ainsi de suite (une opération n-aire dépile n opérandes, leur applique l'opération et empile le résultat)

Exemple de calcul : (">" signale une invite destinée à l'utilisateur)

[1] `https://moodle.u-paris.fr/mod/assign/view.php?id=1145254`

```
> 12
12
> 5
12, 5
> 3
12, 5, 3
> +
12, 8
> *
96
```

Il est utile d'afficher le contenu de la pile à chaque étape, pour permettre à l'utilisateur de voir où il en est. Le sommet de la pile affichée est toujours le résultat de la dernière opération réalisée.

4 Calcul sur les ensembles

Dans ce sujet, les opérandes sont des ensembles d'entiers. Les opérations seront les suivantes :

- Une constante entière (`1`, `-6`, `42`, ...) : créée un singleton contenant cet entier ($\{1\}$, $\{-6\}$, $\{42\}$, ...).
- `U` (union) : opération binaires dont la valeur est l'union de deux ensembles
- Idem pour `I` (intersection).
- `M` (somme de Minkowski) : opération binaire dont la valeur est l'ensemble des entiers qui sont la somme d'un élément de la première opérande et d'un élément de la deuxième opérande ($M(\{1,6\},\{-5,20\}) = \{-4,1,21,26\}$).
- `min` et `max` : opérations unaires retournant respectivement le plus petit et le plus grand élément (sous la forme d'un singleton)

À faire Programmez une calculatrice RPN qui implémente ce calcul d'ensembles en choisissant comme implémentation du type des opérandes, une des implémentations de l'interface `Set` de java. Remarquez qu'une limitation de cette approche est qu'on ne peut que manipuler des ensembles finis.

5 Calculatrice symbolique

Maintenant, on change la façon de faire : le type des opérandes n'est plus les ensembles d'entiers, mais les formules à valeur dans un ensemble d'entiers. Quand l'utilisateur entre une opération n-aire $op(\cdot,\cdot,\dots)$, cela dépile donc n formules et crée une nouvelle formule $op(f_1, f_2, \dots)$, qui sera empilée.
Toute formule est susceptible d'être évaluable en tant qu'ensemble fini (représenté par le type que vous aviez choisi dans la questions précédente).Toute formule a donc une méthode `eval`, qui retourne un ensemble fini d'entiers... ou rien.
Pour permettre d'effectuer ce calcul, on va supposer que toute formule ensembliste possède en plus les méthodes suivantes :

- `min` : retourne l'entier le plus petit de l'ensemble repésenté ou rien
- `max` : retourne l'entier le plus petit de l'ensemble représenté ou rien
- `contains` : dit si l'entier passé en paramètre appartient à l'ensemble représenté.

La présence de ces 3 méthodes garantit de pouvoir évaluer l'expression si l'ensemble représenté est fini (en effet : il est fini si et seulement s'il est borné, et dans ce cas, on peut vérifier tous les éléments entre `min` et `max`). Cela vous permettra d'écrire une implémentation par défaut de la méthode `eval`.

À faire Écrivez une nouvelle calculatrice basée sur ce principe.
Pensez aux `interface`s et aux `record`s (regardez aussi `enum` et `sealed`, sait-on jamais...).
Améliorez l'affichage de la pile, en donnant pour chaque élément à la fois la formule et la valeur de son évaluation. Trouvez un moyen de ne pas refaire plusieurs fois les mêmes calculs.
Ajoutez de nouvelles opérations :

- `UC` : opération unaire qui retourne la fermeture par le haut de son opérande, c'est-à-dire l'ensemble contenant tous les entiers supérieurs ou égaux à l'un des éléments de l'opérande.
- `DC` : idem pour les éléments inférieurs.

Ajoutez une commande annulation (on l'entre par l'invite de commande, comme les opérations, sauf que ce n'est pas une opération). Implémentez-la de façon astucieuse, sans historique : on utilisera le fait que toute formule connait ses sous-formules, qui peuvent donc être remises en pile quand on annule (attention à respecter l'ordre). Faites attention à la façon dont vous allez ajouter cette fonctionnalité ! (nouvelle méthode dans le type formule ? Ou bien fonction extérieure qui gère tous les cas ? Qu'est-ce qui sera le plus pertinent ? Argumentez !)

6 Au delà des ensembles

Les ensembles, `Set` en Java, ne sont qu'un cas particulier de collection. De plus, l'interface `Set` n'introduit pas de méthode qui ne soit pas déjà dans `Collection`.
Il est donc très facile de modifier votre code de la section 4 pour qu'il puisse fonctionner avec d'autres collections (par exemple des listes) avec peu de modifications (seulement les noms des types a priori). La seule différence sera le résultat des calculs.

À faire Transformez votre code de la section 4 pour qu'il puisse fonctionner au choix avec des listes ou des ensembles à l'aide d'un simple réglage (option en ligne de commande). Attention, il ne s'agit pas de mettre ensemble deux versions du projet, mais d'utiliser la généricité.
Vérifiez que cela fonctionne, et remarquez les différences entre les deux modes de fonctionnement.

7 Contraintes imposées

Démonstration/cas de test : pour la fiabilité de votre code, mais aussi pour pouvoir faire une soutenance concluante, prévoyez des cas d'exécution non triviaux démontrant que votre programme fonctionne pour tous les cas prévus.

Observabilité : Les exécutions doivent être visibles. Notamment, faites un affichage de toutes les variables d'état à chaque étape.

8 Hors sujet

On ne demande pas d'interface graphique.

9 Critères d'évaluation

Le rendu :

- Le programme est écrit en Java (si version différente de 21, précisez le dans votre `README.md`).
- Le `README.md` indique comment compiler, exécuter et utiliser votre programme et ses tests.
- Le code respecte les conventions de codage usuelles.
- La compilation selon vos instructions doit terminer sans erreur ni avertissement.
- Le `README.md` décrit les fonctionnalités effectivement implémentées et explique vos choix techniques (modélisation, patrons de conception, ...).

- Le code est architecturé intelligemment en favorisant la réutilisation de code. Notamment, il faut utiliser des patrons de conception vus en cours ainsi que les constructions les plus appropriées fournies par Java.
- L'exécution doit être correcte : elle respecte le cahier des charges et ne quitte pas de façon non contrôlée.
- Les classes et méthodes sont documentées (javadoc).

La soutenance :

- Des démos ont été préparées pour exposer les fonctionnalités du projet.
- Le programme doit pouvoir être compilé et exécuté sous les yeux du jury à partir de la version présente sur la branche principale de votre git (ou bien la dernière version avant la date limite).
- Vous ferez en sorte de pouvoir répondre à toute question concernant vos choix techniques. C'est-à-dire que non seulement vous saurez expliquer ce que vous avez fait, mais aussi ce que vous auriez-pu faire à la place et ce qui vous a mené à faire le choix que vous avez fait. N'hésitez pas à relire le cours et les TPs de CPOO5 en faisant le projet.
- La soutenance portera aussi bien sur votre projet que sur les parties du cours qui vous ont été, ou auraient pu vous être, utiles pour votre projet.